Assignment 1 Report: Symmetric Encryption in a Mobile Banking Application

Group 22 · Repository: `rifaidh/InfoSec-Symmetric-Encryption-Group22`

1. Scenario

We built a digital bank onboarding portal: a Flask web application where customers register for a mobile banking account online. During registration a user enters personal data (full name, NIK, email, phone number, date of birth and address), uploads three documents (a KTP scan as JPEG, PNG or PDF; a supporting office document such as a payslip as PDF, DOCX or XLSX; and a selfie video for e-KYC as MP4 or WEBM), sets a password and consents to data processing under Indonesia's UU PDP No. 27/2022. After logging in, a user can view their own profile and download their own documents, and cannot reach anyone else's data.

The application is useful in this scenario because Indonesian banks must verify customer identity (KYC), which means collecting exactly the data that does the most damage when it leaks. The portal stores ordinary personal data (name, NIK, address), special-category data under UU PDP Pasal 4(2) and GDPR Art. 9 (the biometric selfie video and personal financial documents) and scanned identity documents. A breach could lead to identity theft, financial fraud and a compromised face that can never be changed. GDPR Art. 32(1)(a) names encryption as an appropriate technical safeguard, and UU PDP Pasal 35 requires security measures proportional to the sensitivity of the data. We therefore encrypt every private value before it is saved, so a stolen database or disk image contains only ciphertext.

2. Analysis: Cipher Performance Comparison

Customers notice upload and download latency, while the server feels CPU load when many people register at once. We therefore measured encrypt and decrypt wall-clock time (median of 30 runs after 3 warm-up runs, with the cipher order randomised), throughput in MB/s, storage overhead in bytes and peak memory. All three required ciphers (AES, DES, RC4) were tested, plus the layered mode that stacks all three. Four object sizes stand in for real uploads: a 1 KB profile JSON, a 500 KB KTP scan, a 2 MB office document and a 20 MB selfie video, each with random and with repetitive content.

We chose this analysis because the assignment asks how the ciphers compare in practice, and in an onboarding flow the practical questions are whether a cipher slows the user down (timing and throughput) and whether the server copes under load (memory). The random versus repetitive test checks that no cipher leaks patterns in the data, the weakness that ECB mode would show. We also ran a 200 000 trial test of the RC4 second-byte bias and timed the HKDF key derivation. Table 1 shows the results for the 20 MB selfie video (Python 3.14, Apple ARM, pycryptodome).

Cipher	Encrypt (ms)	Decrypt (ms)	MB/s	One full registration
AES-256-GCM	74	73	271	83 ms
DES-CTR + HMAC	449	448	45	503 ms (6.1× AES)
RC4-drop + HMAC	22	22	902	25 ms (0.3× AES)
Layered AES+DES+RC4	546	544	37	613 ms (7.4× AES)

Table 1: Encryption cost for a 20 MB selfie video.

Speed alone does not decide the choice. Uploading 20 MB over a 10 Mbit/s connection takes about 16 s, so AES adds 0.07 s (under 0.5%) and DES adds 0.45 s (under 3%), which a user will not notice. Security therefore has to be the deciding factor. RC4 is the fastest here because this ARM machine has no AES-NI, but it is prohibited in TLS (RFC 7465) because of bias in its output. Random and repetitive content differ by at most 0.7%, so no cipher showed pattern-dependent behaviour. The RC4 second-byte bias is 2.08× the ideal value without dropping bytes and 1.00× after dropping the first 3 072, which is why we drop them. Storage overhead is between 47 and 156 bytes per object (a 19-byte header and salt plus the nonce and tag: 47 for AES-GCM, 55 for DES, 51 for RC4 and 156 for the layered mode), and peak memory is about three times the object size, so neither is a concern.

3. Design Justifications

Architecture: Flask and SQLite. The scenario is a server-rendered web application with a REST API. Flask is small enough to cover the KYC flow and leaves our effort for the cryptography. SQLite stores metadata, and encrypted files live on disk as `.bin` blobs. All encryption goes through two functions in the hooks module, `protect()` and `unprotect()`: every private value passes through them before saving and after reading, and the cipher is picked there per data type. Application code never calls a cipher directly, so it is hard to forget to encrypt something and the cipher choice lives in one place.

Mode choices, with no ECB.

- *AES-256-GCM:* GCM gives confidentiality and integrity in one pass with no padding (Stallings §12.7). AES-CTR with HMAC was 14% faster, but it makes us combine the encryption and the MAC by hand, while GCM is a standard construction with less room for mistakes. It uses a random 12-byte nonce, and the header, user id and purpose are authenticated as additional data.
- *DES-CTR + HMAC-SHA256:* CTR was the fastest DES mode we measured (447 ms against 521 ms for CBC on 20 MB), needs no padding and can stream. DES has no authenticated mode, so we add encrypt-then-MAC with HMAC-SHA256 and verify the MAC before decrypting anything. The 8-byte block is split into a 4-byte random nonce and a 4-byte counter.
- *RC4 (first 3 072 bytes dropped) + HMAC-SHA256:* RC4 is a stream cipher, so it has no modes. We discard the first 3 072 keystream bytes because of their known bias (Stallings §8.5), which our test confirmed, and HMAC-SHA256 provides integrity in the same way as for DES. RC4 takes no IV, so the keystream is unique only because every object has its own key.
- *ECB* is not used anywhere in the application, the tests or the benchmarks.

Key handling: HKDF per object. A single `MASTER_KEY` from an environment variable is hashed with SHA-256 and expanded with HKDF-SHA256 for every object. The inputs are a fresh random 16-byte salt and a context made of the cipher name, `user_id` and data purpose, so each stored object gets its own key (19 µs per derivation). A key and nonce pair is therefore never reused across objects, which matters for GCM and CTR. Binding to the user and purpose happens twice: the context changes the derived key, and the same values are part of the authenticated data together with the header. A blob copied to another user or document type therefore fails the integrity check, and so does a blob whose header was edited.

Cipher per data type. Ciphers are assigned by how much harm a leak would cause. AES-GCM protects the profile and the selfie video (biometric data and the full identity, the highest harm), RC4 with HMAC protects office documents (financial data, medium harm), and DES-CTR with HMAC protects KTP scans (lowest marginal harm, because the same fields are in the encrypted profile and KTP copies are widely shared). A registration then costs 87 ms against 83 ms with AES only, which is effectively the same. The assignment requires all three ciphers, and this policy uses each of them where its weakness matters least. The policy is a table in the hooks module that can be overridden per data type with `CIPHER_POLICY`, or replaced entirely with `CIPHER=aes`, `des`, `rc4` or `layered`. Each blob records its own cipher in the header, so changing the policy does not make older files unreadable.

Layered mode. With `CIPHER=layered` every object is encrypted with AES, then DES, then RC4, each layer with its own salt and key. It costs 613 ms, 7.4× all-AES, but an attacker who breaks one cipher still faces the other two layers. The key context of each layer includes its position and the total number of layers, so removing or reordering layers makes decryption fail.

Storage: encrypted files on disk, HMAC lookups in the database. Email and NIK are never stored in plaintext. Keyed HMAC digests serve as lookup indices for login and duplicate checks, and because they are keyed, someone holding only the database cannot test guesses of NIK values. Passwords are hashed with Werkzeug's `generate_password_hash` rather than encrypted, because the server never needs to recover them. On download the server recomputes SHA-256 of the decrypted file and rejects it if it differs from the stored value, a second integrity check on top of the cryptographic tag.

Known limitations. DES (56-bit key) and RC4 are cryptographically weak and are used only because the assignment requires all three ciphers; in production, `CIPHER=aes` moves everything to AES-256-GCM. `MASTER_KEY` lives on the application server, so whoever obtains it can derive every object key, and a dedicated key management service would be the next step but is out of scope. If `MASTER_KEY` is not set the code falls back to a development key, so a real deployment must set it.